DỰ ÁN CUỐI KHOÁ

Job Board fullstack

> Một dự án đủ React, API, database, test, Docker, CI và deploy thật, để chứng minh bạn sẵn sàng đi làm và để đưa lên GitHub

Backend chọn Laravel hoặc NestJS, frontend React và TypeScript

Chia thành 9 chặng, mỗi chặng có tiêu chí hoàn thành kiểm tra được

Bạn tự code, tài liệu chỉ đưa yêu cầu và trỏ đến Part cần đọc

Vì sao cần dự án này

Học xong tài liệu mới là biết. Đi làm cần **làm được một sản phẩm từ đầu đến khi chạy thật**. Dự án này gom toàn bộ lộ trình vào một sản phẩm, để bạn:

- Tự kiểm tra những gì còn yếu, vì ghép các phần lại mới lộ ra lỗi.
- Có một dự án thật trên GitHub, có link chạy được và có README rõ ràng.
- Có câu chuyện kỹ thuật để kể khi phỏng vấn: gặp vấn đề gì và xử lý ra sao.

Tài liệu này không có code giải sẵn. Bạn tự code, mỗi chặng trỏ đến Part cần đọc. Kẹt thì quay lại Part đó, hoặc nhờ review sau khi đã thử.

Đề bài: Job Board

Một trang tuyển dụng có bốn nhóm người dùng:

Vai trò	Làm được gì
Khách	Xem danh sách tin, tìm kiếm, lọc, xem chi tiết
Ứng viên	Đăng ký, đăng nhập, lưu tin, nộp hồ sơ kèm file CV, xem trạng thái hồ sơ
Nhà tuyển dụng	Quản lý tin của mình, xem hồ sơ ứng tuyển, đổi trạng thái hồ sơ, trả phí đẩy tin nổi bật
Quản trị	Duyệt tin, khoá tài khoản

Chọn công nghệ

Phần	Lựa chọn
Backend	**Laravel hoặc NestJS.** Chọn theo công việc bạn nhắm tới. Làm xong có thể viết lại bằng cái còn lại để so sánh
Frontend	React + TypeScript + Vite + TanStack Query + React Hook Form + Zod
Database	MySQL hoặc PostgreSQL, không dùng SQLite
Cache, queue	Redis
File	Lưu đĩa khi dev, S3 hoặc dịch vụ tương thích khi production
Chạy	Docker Compose

Giữ cả dự án trong một repo riêng, gồm thư mục `backend/` và `frontend/`.

Thiết kế dữ liệu gợi ý

Bảng	Cột chính	Ghi chú
`users`	`id`, `email` unique, `password`, `role`, `locked_at`	
`companies`	`id`, `owner_id`, `name`, `city`	Thuộc nhà tuyển dụng
`jobs` (đặt tên `job_posts` nếu dùng Laravel)	`id`, `company_id`, `title`, `description`, `type`, `salary_min`, `salary_max`, `status`, `boosted_until`, `published_at`	Index cho `status` + `published_at`
`applications`	`id`, `job_id`, `user_id`, `cv_path`, `status`	**Unique** `(job_id, user_id)`: mỗi người nộp một lần
`saved_jobs`	`user_id`, `job_id`	Bảng trung gian
`payments`	`id`, `job_id`, `amount`, `status`, `provider_ref`	Đẩy tin

Bảng	Cột chính	Ghi chú
`webhook_events`	`provider`, `event_id`	Unique, chống xử lý trùng

Tên bảng `jobs` trùng với bảng queue của Laravel, nên nếu dùng Laravel hãy đặt `job_posts`.

Trường `status` nên dùng enum: tin có `draft`, `pending`, `published`, `closed`; hồ sơ có `submitted`, `reviewing`, `accepted`, `rejected`.

Các chặng

Mỗi chặng có **tiêu chí hoàn thành kiểm tra được**. Chưa đạt thì chưa sang chặng sau. Mỗi chặng nên là một nhánh và một Pull Request, để có lịch sử commit đẹp.

Cột Part dùng ký hiệu: **L** là plan Laravel, **N** là plan NestJS, **R** là plan React, **JS** là `js-crash-course`.

Chặng 1: Khởi tạo và Docker

Tạo repo, `backend`, `frontend`, `compose.yaml` gồm app, database, Redis. Có `.env.example`, `.gitignore`.

Tiêu chí hoàn thành
Máy mới clone về chỉ cần `docker compose up` là chạy được
`GET /api/health` trả 200 và kiểm tra được kết nối database
Không có secret nào trong git

Đọc: L Part SAIL, L Part 22; N Part 25; R Part 01.

Chặng 2: Đăng nhập và phân quyền

Đăng ký, đăng nhập, đăng xuất, refresh token. Ba vai trò, và middleware hoặc guard kiểm tra vai trò.

Tiêu chí hoàn thành
Mật khẩu được hash, không lộ ra trong response hay log
Access token ngắn hạn, refresh token trong cookie `HttpOnly`, có xoay vòng
Test: sai mật khẩu 401, sai quyền 403, chưa đăng nhập 401
Giới hạn số lần đăng nhập sai

Đọc: L Part 09, 19; N Part 10, 17, 27.

Chặng 3: Tin tuyển dụng và tìm kiếm

CRUD tin. Danh sách có phân trang, tìm theo từ khoá, lọc theo loại, thành phố, khoảng lương, sắp xếp. Nhà tuyển dụng chỉ sửa được tin của mình.

Tiêu chí hoàn thành
Sửa `id` trên URL không xem hay sửa được tin của công ty khác (test IDOR)
Không có N+1: kiểm tra bằng số câu query trong test
Có index, và `EXPLAIN` xác nhận truy vấn danh sách dùng index
Cột sắp xếp được kiểm tra theo danh sách cho phép, không nhận trực tiếp từ người dùng

Đọc: L Part 02–08, 12, SQL; N Part 02–09, 12.

Chặng 4: Frontend

Trang danh sách, chi tiết, đăng nhập, quản lý tin, form thêm và sửa.

Tiêu chí hoàn thành
`tsc --noEmit` không lỗi, không dùng `any`
Bộ lọc và trang lưu trên URL: F5 và gửi link vẫn đúng
Có trạng thái đang tải, rỗng, lỗi ở mọi nơi gọi API
Lỗi validate từ server hiện đúng dưới ô nhập
Đăng nhập rồi F5 vẫn giữ phiên, token hết hạn tự làm mới

Đọc: JS toàn bộ; R Part 14–17.

Chặng 5: Nộp hồ sơ và queue

Nộp hồ sơ kèm file PDF. Gửi mail cho nhà tuyển dụng qua queue. Nhà tuyển dụng đổi trạng thái thì ứng viên nhận thông báo.

Tiêu chí hoàn thành
Kiểm tra loại file thật và kích thước, không dùng tên file gốc, file không truy cập công khai được
Bấm nộp hai lần liên tiếp vẫn chỉ có một hồ sơ, nhờ unique index, và trả thông báo rõ ràng
Response trả về ngay, mail được gửi trong queue. Tắt worker thì nộp vẫn thành công
Job có retry, và ghi nhận job thất bại

Đọc: L Part 13, 14, 18, 20, 24; N Part 18, 22, 23, 28.

Chặng 6: Thanh toán đẩy tin và webhook

Giả lập cổng thanh toán: một endpoint nhận webhook có chữ ký. Thanh toán xong, tin được đẩy lên đầu danh sách trong 7 ngày.

Tiêu chí hoàn thành
Sai chữ ký trả 401
Gửi lại cùng một webhook 3 lần, thời hạn đẩy tin chỉ cộng một lần
Test đồng thời: gửi 10 request cùng lúc vẫn đúng

Đọc: L Part 24, 28; N Part 28.

Chặng 7: Tối ưu và realtime

Cache trang chủ, xoá cache khi tin đổi. Thông báo realtime khi hồ sơ đổi trạng thái (tuỳ chọn).

Tiêu chí hoàn thành
Có số đo trước và sau khi cache, ghi vào README
Danh sách code splitting, Lighthouse từ 90 trở lên
Thông báo đến đúng người, người khác không nhận được

Đọc: L Part 16; N Part 20, 29; R Part 10, 20.

Chặng 8: Test và CI

Tiêu chí hoàn thành
Mọi luồng chính ở chặng 2, 3, 5, 6 có feature test hoặc e2e test
Frontend có test cho form và luồng đăng nhập
GitHub Actions chạy lint, kiểm tra kiểu, test, build trên mỗi Pull Request
Test backend chạy trên cùng loại database với production
Nhánh `main` được bảo vệ, phải qua CI mới merge được

Đọc: L Part 11, 26; N Part 11, 30; R Part 13.

Chặng 9: Deploy và README

Tiêu chí hoàn thành
Có link chạy thật, có HTTPS
Migration chạy khi deploy, có cách quay lại khi lỗi
`APP_DEBUG` tắt, không lộ stack trace
Log xem được, có backup database
README đầy đủ theo mục dưới đây

Đọc: L Part 22, SEC; N Part 25; R Part 20.

README đạt chuẩn

Người tuyển dụng thường chỉ dành một đến hai phút. README nên có:

1. Một câu mô tả sản phẩm và **link chạy thật**, kèm tài khoản thử.
2. Ảnh chụp màn hình hoặc GIF ngắn.
3. Sơ đồ kiến trúc đơn giản.
4. Công nghệ, và **lý do chọn** ở những chỗ quan trọng.
5. Các vấn đề kỹ thuật đã xử lý: chống nộp trùng, webhook một lần, refresh token, số liệu cache.
6. Cách chạy bằng một lệnh.
7. Link tài liệu API.
8. Những gì sẽ làm tiếp nếu có thêm thời gian.

Dùng dự án này khi phỏng vấn

Chuẩn bị sẵn câu trả lời theo STAR cho những câu sau, dùng đúng chuyện đã xảy ra trong dự án:

Câu hỏi	Lấy từ
Vấn đề kỹ thuật khó nhất bạn từng xử lý?	Chặng 5 hoặc 6
Làm sao tránh xử lý trùng?	Unique index, webhook
Bạn bảo mật ứng dụng thế nào?	Chặng 2, IDOR ở chặng 3, file ở chặng 5
Bạn tối ưu hiệu năng thế nào, kết quả ra sao?	Số đo ở chặng 7
Bạn test và deploy thế nào?	Chặng 8, 9
Nếu làm lại, bạn sẽ làm khác gì?	Tự ghi lại trong lúc làm

Ghi nhật ký trong lúc làm. Mỗi khi gặp lỗi mất hơn 30 phút, ghi lại triệu chứng, nguyên nhân, cách sửa. Đó chính là những câu chuyện tốt nhất khi phỏng vấn, và sau vài tháng bạn sẽ không còn nhớ nếu không ghi.

Tự đánh giá

Đạt phần lớn các mục dưới đây thì bạn đã sẵn sàng cho vị trí junior, và có cơ sở để đòi mức lương cao hơn:

- Tự dựng được dự án từ đầu, không cần làm theo tutorial.
- Giải thích được **vì sao** chọn từng cách làm, và đánh đổi của nó.
- Đọc log và tự tìm được nguyên nhân lỗi.
- Có test, và tin vào test khi sửa code.
- Đã deploy thật, và đã từng xử lý lỗi trên môi trường thật.
- Đọc được code người khác viết và review được Pull Request.